

FUZHOU UNIVERSITY

- ## ① LALR(1) 分析器

- ## ② 二义性文法

- ### ③ 期中复习

- ## ① LALR(1) 分析器

心集，并将它们合并成一个项集。
态数，从而可能得到 LALR 分析表；
此时文法不是 LALR)；

项目集的第一分量集合) 的项集，即除展望符

- LALR 分析的核心思想：寻找同心集，并将它们合并成一个项集。
 - 可以压缩 LR(1) 分析表中的状态数，从而可能得到 LALR 分析表；
 - 压缩可能导致归约/归约冲突 (此时文法不是 LALR)；
 - 压缩不会导致移入/归约冲突。
- **同心项**：具有相同**核心**（LR(1) 项目集的第一分量集合）的项集，即除展望符外，两个 LR(1) 项集是相同的。

考虑文法

$$\begin{aligned} S' &\rightarrow S & (0) \\ S &\rightarrow CC & (1) \\ C &\rightarrow cC & (2) \\ C &\rightarrow d & (3) \end{aligned}$$

- 1 构造 LR(1) 项集族、分析表。
- 2 找出并合并同心项。
- 3 构造新的项集族、分析表。

计算项集族:

状态编码	项集	后继符号	后继状态
l_7	$\{[C \rightarrow d \cdot, \$]\}$	$\#C \rightarrow d$	
l_8	$\{[C \rightarrow cC \cdot, c/d]\}$	$\#C \rightarrow cC$	
l_9	$\{[C \rightarrow cC \cdot, \$]\}$	$\#C \rightarrow cC$	

构建 LR(1) 分析表如下:

状态	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

状态编码	项集	后继符号	后继状态
I_0	$\{[S' \rightarrow \cdot S, \$],$	S	I_1
	$[S \rightarrow \cdot CC, \$],$	C	I_2
	$[C \rightarrow \cdot cC, c/d]$	c	I_3
	$[C \rightarrow \cdot d, c/d]\}$	d	I_4
I_1	$\{[S' \rightarrow S \cdot]\}$	$\#S' \rightarrow S$	
I_2	$\{[S \rightarrow C \cdot C, \$],$	C	I_5
	$[C \rightarrow \cdot cC, \$]$	c	I_6
	$[C \rightarrow \cdot d, \$]\}$	d	I_7
I_3	$\{[C \rightarrow c \cdot C, c/d],$	C	I_8
	$[C \rightarrow \cdot cC, c/d],$	c	I_3
	$[C \rightarrow \cdot d, c/d]\}$	d	I_4
I_4	$\{[C \rightarrow d \cdot, c/d]\}$	$\#C \rightarrow d$	
I_5	$\{[S \rightarrow CC \cdot, \$]\}$	$\#S \rightarrow CC$	
I_6	$\{[C \rightarrow c \cdot C, \$],$	C	I_9
	$[C \rightarrow \cdot cC, \$],$	c	I_6
	$[C \rightarrow \cdot d, \$]\}$	d	I_7

状态编码	项集	后继符号	后继状态
I_7	$\{[C \rightarrow d \cdot, \$]\}$	$\#C \rightarrow d$	
I_8	$\{[C \rightarrow cC \cdot, c/d]\}$	$\#C \rightarrow cC$	
I_9	$\{[C \rightarrow cC \cdot, \$]\}$	$\#C \rightarrow cC$	

① 先看单独的可归约状态， I_8 与 I_9 仅是同心项，先合并为 I_{89} ；

① 先看单独的可归约状态, l_8 与 l_9 仅是同心项, 先合并为 l_{89} ;

状态编码	项集	后继符号	后继状态
l_0	$\{[S' \rightarrow \cdot S, \$],$	S	l_1
	$[S \rightarrow \cdot CC, \$],$	C	l_2
	$[C \rightarrow \cdot cC, c/d]$	c	l_3
	$[C \rightarrow \cdot d, c/d]\}$	d	l_4
l_1	$\{[S' \rightarrow S \cdot]\}$	$\#S' \rightarrow S$	
l_2	$\{[S \rightarrow C \cdot C, \$],$	C	l_5
	$[C \rightarrow \cdot cC, \$]$	c	l_6
	$[C \rightarrow \cdot d, \$]\}$	d	l_7
l_3	$\{[C \rightarrow c \cdot C, c/d],$	C	l_{89}
	$[C \rightarrow \cdot cC, c/d],$	c	l_3
	$[C \rightarrow \cdot d, c/d]\}$	d	l_4
l_4	$\{[C \rightarrow d \cdot, c/d]\}$	$\#C \rightarrow d$	
l_5	$\{[S \rightarrow CC \cdot, \$]\}$	$\#S \rightarrow CC$	
l_6	$\{[C \rightarrow c \cdot C, \$],$	C	l_{89}
	$[C \rightarrow \cdot cC, \$],$	c	l_6
	$[C \rightarrow \cdot d, \$]\}$	d	l_7

① 先看单独的可归约状态, l_8 与 l_9 仅搜索符不同, 是同心项, 先合并为 l_{89} ;

状态编码	项集	后继符号	后继状态
I_7	$\{[C \rightarrow d \cdot, \$]\}$	$\#C \rightarrow d$	
I_{89}	$\{[C \rightarrow cC \cdot, c/d/\$]\}$	$\#C \rightarrow cC$	

- ① 先看单独的可归约状态, l_8 与 l_9 仅搜索符不同, 是同心项, 先合并为 l_{89} ;
- ② 类似地, l_4 与 l_7 也是同心项, 合并为 l_{47} ;

状态编码	项集	后继符号	后继状态
I_{47}	$\{[C \rightarrow d, c/d/\$]\}$	$\#C \rightarrow d$	
I_{89}	$\{[C \rightarrow cC, c/d/\$]\}$	$\#C \rightarrow cC$	

- 1 先看单独的可归约状态, l_8 与 l_9 仅搜索符不同, 是同心项, 先合并为 l_{89} ;
- 2 类似地, l_4 与 l_7 也是同心项, 合并为 l_{47} ;

状态编码	项集	后继符号	后继状态
l_0	$\{[S' \rightarrow \cdot S, \$],$	S	l_1
	$[S \rightarrow \cdot CC, \$],$	C	l_2
	$[C \rightarrow \cdot cC, c/d]$	c	l_{36}
	$[C \rightarrow \cdot d, c/d]\}$	d	l_{47}
l_1	$\{[S' \rightarrow S \cdot]\}$	$\#S' \rightarrow S$	
l_2	$\{[S \rightarrow C \cdot C, \$],$	C	l_5
	$[C \rightarrow \cdot cC, \$]$	c	l_{36}
	$[C \rightarrow \cdot d, \$]\}$	d	l_{47}
l_5	$\{[S \rightarrow CC \cdot, \$]\}$	$\#S \rightarrow CC$	
l_{36}	$\{[C \rightarrow c \cdot C, c/d/\$],$	C	l_{89}
	$[C \rightarrow \cdot cC, c/d/\$],$	c	l_{36}
	$[C \rightarrow \cdot d, c/d/\$]\}$	d	l_{47}

- 1 先看单独的可归约状态, l_8 与 l_9 仅搜索符不同, 是同心项, 先合并为 l_{89} ;
- 2 类似地, l_4 与 l_7 也是同心项, 合并为 l_{47} ;
- 3 最后发现 l_3 与 l_6 仅搜索符不同, 同样文法符号的后继状态也相同, 故 l_3 与 l_6 是同心项, 合并二者为 l_{36} 。

LALR 分析表

状态	ACTION			GOTO	
	c	d	\$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

LR(1) 分析表

状态	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

重标号后的 LALR 分析表

状态	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

重标号后的 LALR 分析表

状态	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

重标号后的 LALR 分析表

状态	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

重标号后的 LALR 分析表

状态	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

有区别的例子

例

考虑文法

$$S' \rightarrow S \quad (0)$$
$$S \rightarrow Bbb \quad (1)$$
$$S \rightarrow aab \quad (2)$$
$$S \rightarrow bBa \quad (3)$$
$$B \rightarrow a \quad (4)$$
$$\text{FOLLOW}(B) = \{a, b\}$$
$$l_0 : S' \rightarrow \cdot S, \$$$
$$S \rightarrow \cdot Bbb, \$$$
$$S \rightarrow \cdot aab, \$$$
$$S \rightarrow \cdot bBa, \$$$
$$B \rightarrow \cdot a, b$$
$$l_1 : S' \rightarrow S, \$$$
$$l_2 : B \rightarrow \cdot bb, \$$$
$$l_3 : S \rightarrow a \cdot ab, \$$$
$$B \rightarrow a \cdot, b$$

如果是 SLR 方法，由于 $a \in \text{FOLLOW}(B)$ ，故会产生移入于归约冲突。

而在 LR(1) 与 LALR 方法中，这就不会产生冲突。

LR(1) 项集:

考慮文法

$$C \rightarrow e \quad (6)$$
$$C \rightarrow \cdot e, d$$
$$I_7 : S \rightarrow bC \cdot c, \$$$
$$l_{13} : S \rightarrow bBd., \$$$

LR(1) 项集:

考慮文法

$$C \rightarrow e \quad (6)$$
$$C \rightarrow \cdot e, d$$
$$I_7 : S \rightarrow bC \cdot c, \$$$
$$l_{13} : S \rightarrow bBd., \$$$

LR(1) 项集:

考慮文法

$$C \rightarrow e \quad (6)$$
$$C \rightarrow \cdot e, d$$
$$I_7 : S \rightarrow bC \cdot c, \$$$
$$C \rightarrow e., c/d$$

LALR 表怎么建

- 1 暴力建：先构建规范 LR(1) 项集族，然后从最简单的单归约项开始逐步合并同心集，最终得到 LALR 的项集族并构建分析表；
- 2 基于 LR(0) 进行构建：
 - 1 使用 DeRemer-Pennello 算法，通过两次遍历（向前传播和反向传播）高效计算向前看符号；
 - 2 使用位向量（Bitmap）优化符号集合；
 - 3 惰性求值（Lazy Evaluation）：仅在需要时生成特定状态的向前看符号，减少冗余计算。

[illegible]

步骤	LR(0)	SLR(1)	LR(1)	LALR(1)
构建增广文法	✓	✓	✓	✓
对所有规则编号	✓	✓	✓	✓
求非终结符的 FOLLOW 集		✓		
求非终结符的 FIRST 集			✓	✓
构建 LR(0) 项集族	✓	✓		
构建 LR(1) 项集族			✓	✓
合并同心集				✓
根据项集族构建分析表	✓	✓	✓	✓

LR(0)	SLR(1)	LR(1)	LALR(1)
填满行	看 FOLLOW 集合	看搜索符/展望符	

- ## ① LALR(1) 分析器

二义性文法

- 二义文法不是 LR 类文法（也非 LL 文法）。
- 某些二义文法很容易理解，可以在构建 LR 分析器时进行特殊处理：
 - 例如可以通过人为规定优先级和结合性（解决冲突），来构建 LR 分析器。
- 应该保守地使用二义性文法，并且必须在严格控制之下使用，因为稍有不慎就会导致语法分析器所识别的语言出现偏差。

① LALR(1) 分析器

② 二义性文法

③ 期中复习

编译程序概论

高级语言的构成

形式化文法

词法分析与正则文法

语法分析与上下文无关文法

语言鸿沟：硬件只能**直接**理解和执行机器语言，而人类理解和编写，程序通常使用高级语言（C/C++、Java等）。

- ¹ 比如国产硬件平台

为什么需要编译程序?

- 1 **编译程序消除了人类与机器之间的语言鸿沟**：硬件只能**直接**理解和执行机器语言（即二进制指令）。而为了方便人类理解和编写，程序通常使用高级语言（如 C、C++、Java 等）开发。
- 2 **编译程序可以优化程序性能**：机器代码的执行效率直接影响程序的性能。通过编译器的优化，程序运行速度更快，资源利用率更高，这对性能敏感的应用（如游戏、科学计算、嵌入式系统）尤为重要。

¹ 比如国产硬件平台

为什么需要编译程序?

- 1 **编译程序消除了人类与机器之间的语言鸿沟**：硬件只能**直接**理解和执行机器语言（即二进制指令）。而为了方便人类理解和编写，程序通常使用高级语言（如 C、C++、Java 等）开发。
- 2 **编译程序可以优化程序性能**：机器代码的执行效率直接影响程序的性能。通过编译器的优化，程序运行速度更快，资源利用率更高，这对性能敏感的应用（如游戏、科学计算、嵌入式系统）尤为重要。
- 3 **静态错误检测可以避免出现低级错误**：编译程序帮助开发者在程序运行之前发现并修复错误，减少了运行时错误的可能性，提高了程序的可靠性和开发效率。

¹ 比如国产硬件平台

为什么需要编译程序?

- 1 **编译程序消除了人类与机器之间的语言鸿沟**：硬件只能**直接**理解和执行机器语言（即二进制指令）。而为了方便人类理解和编写，程序通常使用高级语言（如 C、C++、Java 等）开发。
- 2 **编译程序可以优化程序性能**：机器代码的执行效率直接影响程序的性能。通过编译器的优化，程序运行速度更快，资源利用率更高，这对性能敏感的应用（如游戏、科学计算、嵌入式系统）尤为重要。
- 3 **静态错误检测可以避免出现低级错误**：编译程序帮助开发者在程序运行之前发现并修复错误，减少了运行时错误的可能性，提高了程序的可靠性和开发效率。
- 4 **编译程序方便了跨平台开发**¹：不同的计算机硬件和操作系统可能使用不同的指令集和运行环境。编译程序使程序可以在不同的硬件和操作系统上编译并运行，提高了代码的可移植性和跨平台兼容性。

¹ 比如国产硬件平台

处理源代码中的预处理指令（如 `#include`、

- ❶ 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);

C 语言编译器的典型阶段

- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);
- ② 编译阶段 (Compiling): “编”即整理,“译”即翻译。分析高级语言程序并翻译成汇编指令并添加必要的辅助代码、标记等;

C 语言编译器的典型阶段

- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);
- ② 编译阶段 (Compiling): “编”即整理, “译”即翻译。分析高级语言程序并翻译成汇编指令并添加必要的辅助代码、标记等;
- ③ 汇编阶段 (Assembling): 将汇编指令翻译为二进制机器代码, 计算相对跳转位置等, 生成二进制目标文件;

C 语言编译器的典型阶段

- ① 预处理阶段 (Preprocessing): 处理源代码中的预处理指令 (如 `#include`、`#define` 等);
- ② 编译阶段 (Compiling): “编”即整理, “译”即翻译。分析高级语言程序并翻译成汇编指令并添加必要的辅助代码、标记等;
- ③ 汇编阶段 (Assembling): 将汇编指令翻译为二进制机器代码, 计算相对跳转位置等, 生成二进制目标文件;
- ④ 链接阶段 (Linking): 将一个或多个二进制目标文件进行关联, 处理外部函数跳转地址, 全局变量初始化等以生成可执行文件。

编译阶段

- ① 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
- 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。

编译阶段

- 1 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- 2 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。

编译阶段

- 1 词法分析 (Lexical Analysis): 将预处理后的源代码分解为一个个词法单元 (Token)。
 - 过程: 1) 忽略空白字符和注释; 2) 识别关键字、标识符、常量、运算符等。
 - 输出: 词法单元序列。
- 2 语法分析 (Syntax Analysis): 根据语言的语法规则 (文法), 将词法单元序列解析为抽象语法树 (Abstract Syntax Tree, AST)。
 - 过程: 检查语法是否正确, 构建语法树来表示程序的结构。
 - 输出: 抽象语法树。
- 3 语义分析阶段 (Semantic Analysis): 检查程序的语义是否正确, 例如类型检查、变量作用域分析等。
 - 过程: 检查变量是否声明, 检查类型是否匹配, 构建符号表 (Symbol Table), 记录变量、函数等的定义和属性。
 - 经过语义检查和修饰的抽象语法树。

de Generation)：将抽象语法树转换为中间代码。中间代码是一种与硬件无关的低级表示形式，用于代码优化和目标代码生成。

- ◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

编译阶段 (续)

- ④ 中间代码生成 (Intermediate Code Generation): 将抽象语法树转换为中间代码 (Intermediate Representation, IR)。中间代码是一种与硬件无关的低级表示形式 (如三地址码), 以便于后续优化和目标代码生成。
- ⑤ 优化阶段 (Optimization): 对中间代码进行优化, 提高程序的运行效率。
- 过程: 常量折叠、公共子表达式消除、循环优化等。
 - 分类: 前端优化 (与语言相关) 和后端优化 (与硬件相关)。
 - 输出: 优化后的中间代码。

编译阶段 (续)

- 4 中间代码生成 (Intermediate Code Generation): 将抽象语法树转换为中间代码 (Intermediate Representation, IR)。中间代码是一种与硬件无关的低级表示形式 (如三地址码), 以便于后续优化和目标代码生成。
- 5 优化阶段 (Optimization): 对中间代码进行优化, 提高程序的运行效率。
 - 过程: 常量折叠、公共子表达式消除、循环优化等。
 - 分类: 前端优化 (与语言相关) 和后端优化 (与硬件相关)。
 - 输出: 优化后的中间代码。
- 6 目标代码生成 (Code Generation): 将优化后的中间代码转换为目标代码 (机器语言)。
 - 过程: 处理寄存器分配、指令选择等。
 - 输出: 目标代码 (如汇编、Java 字节码、Python 字节码等)。

① LALR(1) 分析器

② 二义性文法

③ 期中复习

编译程序概论

高级语言的构成

形式化文法

词法分析与正则文法

语法分析与上下文无关文法

高级语言的构成

编程语言通常由以下几个核心部分构成：

- ① 语法 (Syntax): 语法是编程语言的结构规则, 定义了程序如何书写。它是语言的“形式”, 规定了哪些字符、符号和结构是合法的。

例

C 语言使用分号 (;) 表示一般语句的结束, 使用 {} 来表示语句块 (序列)。

- ② 语义 (Semantics): 语义是编程语言的“意义”，定义了程序的行为。它描述了程序如何被执行，以及每条语句的具体含义。

例

a=1 在 C 语言中的意义是将 1 赋值给 a，而在 Pascal 语言中则是判断 a 与 1 是否相等。

高级语言的构成 (续)

- ③ 运行库 (Runtime Library): 运行库是编程语言提供的一组预定义函数和模块, 用于支持程序的运行。它通常包含输入输出、内存管理、数学运算等功能。

例

C 语言在 `stdio.h` 头文件中提供了 `printf` 函数来实现格式化输出。

Java 语言则需要使用 `System.out.printf`。

Python 则通过格式化字符串 f-string 功能进行处理。

Python 封装了统一的 socket 接口来处理网络通信，而 C 语言根据操作系统不同使用 socket 或者 winsock 来支持调用不同操作系统的网络通信模块。

① LALR(1) 分析器

② 二义性文法

③ 期中复习

编译程序概论

高级语言的构成

形式化文法

词法分析与正则文法

语法分析与上下文无关文法

语法与语言

在人类社会中，**语言**是一种社会现象，是人类用于交流思想、表达情感和传递信息的符号系统。**语法**是语言中对词汇、句子结构等规则的总结。

在形式语言理论中，**文法 (Grammar)** 和 **语言 (Language)** 是两个核心概念，它们之间的关系概括为：**文法是语言的生成规则，而语言是文法生成的字符串集合。**

基于乔姆斯基的层级理论 (Chomsky Hierarchy), 按照生成能力从**强**到**弱**分成四类:

文法类型	文法名称	生成的语言	对应的自动机	例子
0 型文法	无限制文法	递归可枚举语言	图灵机	$a^n b^n c^n$
1 型文法	上下文相关文法	上下文有关语言	线性有界自动机	$a^n b^n c^n$
2 型文法	上下文无关文法	上下文无关语言	下推自动机	$a^n b^n$
3 型文法	正则文法	正则语言	有限状态自动机	$a^* b^*$

文法的形式化定义

文法的形式化定义为一个四元组 G :

$$\mathcal{G} = (V_T, V_N, S, P).$$

其中，

- V_T 是终结符号集合，可以理解为最终字符，其中有一个特殊符号 ε ，用于表示空；
- V_N 是非终结符号集合，非终结符号也称**文法符号**；
- S 是初始符号；
- P 是产生式/推导式的集合。

文法的产生式

产生式是形如 $\alpha A \rightarrow B\gamma$ 的式子。**注意，产生式的左侧必须要有一个非终结符。**语言推导的过程就是从初始符号开始，每次用一个产生式右边字符串替换左边字符串，最终生成期望字符串的过程。

不同类型语法对产生式的约束如下:

- 无限制文法：没有约束；
- 上下文相关文法：左边的符号串长度不超过右边的符号串长度，如 $\alpha A \beta \rightarrow \alpha \gamma \beta$ ；
- 上下文无关文法：产生式左边必须是一个非终结符，右边可以是任意字符串，如 $A \rightarrow aBa$ ；
- 正则文法的产生式规则必须满足以下两种形式之一：
 - ① 右正则形式：形如 $A \rightarrow aB$ 或 $A \rightarrow a$ ，其中 A 和 B 是非终结符， a 是终结符。
 - ② 左正则形式：形如 $A \rightarrow Ba$ 或 $A \rightarrow a$ ，其中 A 和 B 是非终结符， a 是终结符。

① LALR(1) 分析器

② 二义性文法

③ 期中复习

编译程序概论

高级语言的构成

形式化文法

词法分析与正则文法

语法分析与上下文无关文法

词法分析

- **词法分析**是编译器设计中的**第一个阶段**，其目的是将源代码分解为一个个**词法单元 (Token)**，并忽略无关的字符（如空白字符和注释）。
- **词法分析器 (Lexer)** 是编译器的核心组件之一，它为后续的语法分析和语义分析提供基础。
- 独立词法分析阶段的目的：
 - ① 模块化设计：简化编译器结构、提高可维护性、复用性；
 - ② 提高效率：预处理、快速错误检测、合并重复标识符节约存储空间；
 - ③ 简化后续阶段：词法分析器将源代码抽象为词法单元序列，使得语法分析器可以专注于语言的结构，而不需要处理字符级别的细节。

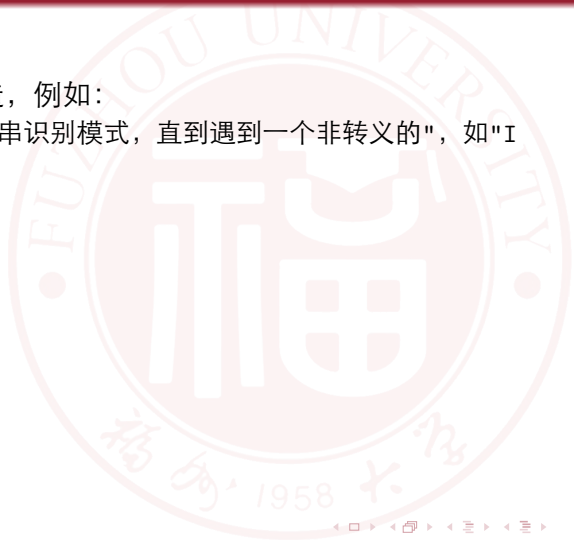
词法分析器的构造

- ① 基于首字符的状态机的手工构造，例如：



词法分析器的构造

- ① 基于首字符的状态机的手工构造，例如：
 - 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\"";



，例如：
串识别模式，直到遇到一个非转义的"，如"I
数字常量识别模式，可处理带前缀和后缀的数字常
，100ULL,0.345L；

- ① 基于首字符的状态机的手工构造，例如：
- 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\";
 - 在一般模式下遇到 0-9 进入数字常量识别模式，可处理带前缀和后缀的数字常量，如 0x1234,0777,0b1010,100ULL,0.345L;

词法分析器的构造

① 基于首字符的状态机的手工构造，例如：

- 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\";
- 在一般模式下遇到 0-9 进入数字常量识别模式，可处理带前缀和后缀的数字常量，如 0x1234,0777,0b1010,100ULL,0.345L;
- 在一般模式下遇到运算符，判断是否需要处理运算符的合并，如 ==,+=,<=;

词法分析器的构造

① 基于首字符的状态机的手工构造，例如：

- 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\";
- 在一般模式下遇到 0-9 进入数字常量识别模式，可处理带前缀和后缀的数字常量，如 0x1234,0777,0b1010,100ULL,0.345L;
- 在一般模式下遇到运算符，判断是否需要处理运算符的合并，如 ==,+=,<=;
- 在一般模式下遇到 #,/,/* 等进入注释处理模式;

词法分析器的构造

① 基于首字符的状态机的手工构造，例如：

- 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\";
- 在一般模式下遇到 0-9 进入数字常量识别模式，可处理带前缀和后缀的数字常量，如 0x1234,0777,0b1010,100ULL,0.345L;
- 在一般模式下遇到运算符，判断是否需要处理运算符的合并，如 ==,+=,<=;
- 在一般模式下遇到 #,/,/* 等进入注释处理模式;
- 处理标识符并记录生成符号表;

词法分析器的构造

① 基于首字符的状态机的手工构造，例如：

- 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\";
- 在一般模式下遇到 0-9 进入数字常量识别模式，可处理带前缀和后缀的数字常量，如 0x1234,0777,0b1010,100ULL,0.345L;
- 在一般模式下遇到运算符，判断是否需要处理运算符的合并，如 ==,+=,<=;
- 在一般模式下遇到 #,/,/* 等进入注释处理模式;
- 处理标识符并记录生成符号表;
- 记录行首空格，例如 Python 语言以此作为语句块的识别;

词法分析器的构造

① 基于首字符的状态机的手工构造，例如：

- 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\";
- 在一般模式下遇到 0-9 进入数字常量识别模式，可处理带前缀和后缀的数字常量，如 0x1234,0777,0b1010,100ULL,0.345L;
- 在一般模式下遇到运算符，判断是否需要处理运算符的合并，如 ==,+=,<=;
- 在一般模式下遇到 #,/,/* 等进入注释处理模式;
- 处理标识符并记录生成符号表;
- 记录行首空格，例如 Python 语言以此作为语句块的识别;
- 记录每个 token 出现在源代码中的位置，便于错误处理和提示;

词法分析器的构造

- ### ① 基于首字符的状态机的手工构造，例如：
- 在一般模式下遇到" 进入字符串识别模式，直到遇到一个非转义的"，如"I said,\"Hello!\";
 - 在一般模式下遇到 0-9 进入数字常量识别模式，可处理带前缀和后缀的数字常量，如 0x1234,0777,0b1010,100ULL,0.345L;
 - 在一般模式下遇到运算符，判断是否需要处理运算符的合并，如 ==,+=,<=;
 - 在一般模式下遇到 #,/,/* 等进入注释处理模式;
 - 处理标识符并记录生成符号表;
 - 记录行首空格，例如 Python 语言以此作为语句块的识别;
 - 记录每个 token 出现在源代码中的位置，便于错误处理和提示;
- ### ② 基于正则文法/正则表达式的自动生成;

正则文法、正则表达式、正则语言

- **正则文法**是生成**正则语言**的一种形式化规则。每个正则文法都对应一个正则语言，反之亦然。
- **正则表达式**是描述正则语言的一种**紧凑表示方法**。每个正则表达式都对应一个正则语言，反之亦然。
- 正则文法和正则表达式可以**相互转换**。例如，任何正则文法都可以转换为**等价**的正则表达式，反之亦然。
- 这三者共同构成了形式语言理论中正则语言的基础，广泛应用于词法分析、模式匹配和有限状态机等领域。

式的表达式，它可以表示正则语言。正则表达式

- **终结符**：表示单个字符（如 a、b）。

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

式的表达式，它可以表示正则语言。正则表达式
b)。
如 ab 表示 a 后跟 b)。

- 终结符：表示单个字符（如 a、b）。
- 连接：表示两个表达式的连接（如 ab 表示 a 后跟 b）。

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

正则表达式与正则语言

正则表达式是一种用于描述字符串模式的表达式，它可以表示正则语言。正则表达式由以下基本元素组成：

- 终结符：表示单个字符（如 a、b）。
- 连接：表示两个表达式的连接（如 ab 表示 a 后跟 b）。
- 并集：表示两个表达式的并集（用 | 表示，如 $a|b$ 表示 a 或 b）²。

²用“|”表示“或”是 IEEE 的标准，“+”是大学在教学时经常使用的，因为这个操作符合我们对加法朴素的印象——结合律、交换律、有“零”元 ($\emptyset$)

正则表达式与正则语言

正则表达式是一种用于描述字符串模式的表达式，它可以表示正则语言。正则表达式由以下基本元素组成：

- 终结符：表示单个字符（如 a、b）。
- 连接：表示两个表达式的连接（如 ab 表示 a 后跟 b）。
- 并集：表示两个表达式的并集（用 | 表示，如 a|b 表示 a 或 b）²。
- 闭包：表示零次或多次重复（用 * 表示，如 a^* 表示零个或多个 a）。

²用“|”表示“或”是 IEEE 的标准，“+”是大学在教学时经常使用的，因为这个操作符合我们对加法朴素的印象——结合律、交换律、有“零”元 ($\emptyset$)

正则表达式与正则语言

正则表达式是一种用于描述字符串模式的表达式，它可以表示正则语言。正则表达式由以下基本元素组成：

- 终结符：表示单个字符（如 a、b）。
- 连接：表示两个表达式的连接（如 ab 表示 a 后跟 b）。
- 并集：表示两个表达式的并集（用 | 表示，如 a|b 表示 a 或 b）²。
- 闭包：表示零次或多次重复（用 * 表示，如 a* 表示零个或多个 a）。
- 可选：表示零次或一次出现（用 ? 表示，如 a? 表示零个或一个 a）。

²用“|”表示“或”是 IEEE 的标准，“+”是大学在教学时经常使用的，因为这个操作符合我们对加法朴素的印象——结合律、交换律、有“零”元 ($\emptyset$)

正则表达式与正则语言

正则表达式是一种用于描述字符串模式的表达式，它可以表示正则语言。正则表达式由以下基本元素组成：

- 终结符：表示单个字符（如 a、b）。
- 连接：表示两个表达式的连接（如 ab 表示 a 后跟 b）。
- 并集：表示两个表达式的并集（用 | 表示，如 a|b 表示 a 或 b）²。
- 闭包：表示零次或多次重复（用 * 表示，如 a* 表示零个或多个 a）。
- 可选：表示零次或一次出现（用 ? 表示，如 a? 表示零个或一个 a）。
- 范围：表示字符范围（如 [a-z] 表示任意一个小写字母）。

²用“|”表示“或”是 IEEE 的标准，“+”是大学在教学时经常使用的，因为这个操作符合我们对加法朴素的印象——结合律、交换律、有“零”元 ($\emptyset$)

正则表达式示例

例

- a^* 表示零个或多个 a 。
- $(a|b)^*$ 表示任意长度的 a 和 b 的组合。
- ab^*c 表示 a 后跟零个或多个 b ，再跟一个 c 。
- $a^+ = aa^*$ ，表示一个或多个 a 。

正则语言

正则语言 (Regular Language) 是由正则文法或正则表达式定义的语言。正则语言具有以下性质:

- 有限性：正则语言可以用有限自动机（如 DFA 或 NFA）来识别。
- 封闭性：正则语言在并集、交集、补集、连接和闭包等操作下是封闭的。
- 可判定性：正则语言的许多性质（如是否为空、是否有限等）是可判定的。

练习

- 写一个 ab 的正则表达式，其所定义的语言不包含“ abb ”子串。

定义的语言不包含“*abb*”子串。

定义的语言不包含“*abb*”的子序列。

- 写一个 ab 的正则表达式, 其所定义的语言不包含 “ abb ” 子串。
- 写一个 ab 的正则表达式, 其所定义的语言不包含 “ abb ” 的子序列。

练习

- 写一个 ab 的正则表达式，其所定义的语言不包含 “ abb ” 子串。
- 写一个 ab 的正则表达式，其所定义的语言不包含 “ abb ” 的子序列。
- 写一个 ab 的正则表达式，其所定义的语言中所有串恰好有偶数个 a 和偶数个 b 。
- 写一个 ab 的正则表达式，其所定义的语言中所有串恰好有奇数个 a 和偶数个 b 。

语言的运算

例

已知 $L = \{A, B, C, D\}$, $M = \{1, 2, 3\}$

- $L \cup M = \{A, B, C, D, 1, 2, 3\}$
- $LM = \{A1, A2, A3, B1, B2, B3, C1, C2, C3, D1, D2, D3\}$
- $L^2 = \{AA, AB, AC, AD, BA, BB, BC, BD, CA, \dots DD\}$
- $L^* = \{\text{所有 ABCD 组成的串, 包含 } \varepsilon\}$
- $L^+ = L^* - \{\varepsilon\}$
- $L(L \cup M) = ?$
- $L(L \cup M)^* = ?$

例

选择所描述语言与 $(0|1)^*1(0|1)^*$ 相同的正规式 (多选)

- 1 $(01|11)^*(0|1)^*$
- 2 $(0|1)^* (10|11|1)(0|1)^*$
- 3 $(1|0)^* 1(1|0)^*$
- 4 $(0|1)^*(0|1)(0|1)^*$

例

选择所描述语言与 $(0|1)^*1(0|1)^*$ 相同的正规式 (多选)

- 1 $(01|11)^*(0|1)^*$
- 2 $(0|1) * (10|11|1)(0|1)^*$
- 3 $(1|0)^*1(1|0)^*$
- 4 $(0|1)^*(0|1)(0|1)^*$

例

选择所描述语言与 $(0|1)^*1(0|1)^*$ 相同的正规式 (多选)

- 1 $(01|11)^*(0|1)^*$
- 2 $(0|1)^*(10|11|1)(0|1)^*$
- 3 $(1|0)^*1(1|0)^*$
- 4 $(0|1)^*(0|1)(0|1)^*$

如何基于正则表达式自动化构造词法分析器

- ① 给出每个子模式（词法单元）的正则表达式 $r_1, r_2, \dots, r_n$;
- ② 构造识别每个子模式非确定性有穷自动机（Non-deterministic Finite Automata, NFA）;
- ③ 集成各子 NFA 得最终 NFA;
- ④ NFA 转 DFA（Deterministic Finite Automata, 确定性有穷状态自动机），并最小化 DFA（合并等价状态）;
- ⑤ 生成该 DFA 的状态转换表和控制执行程序。

有穷状态自动机

- 有穷自动机 (Finite Automata, FA) 由两位神经物理学家 McCulloch 和 Pitts 于 1948 年首先提出, 是对一类处理系统建立的**数学模型**;
- 这类系统具有一系列**离散**的输入输出信息和**有穷**数目的内部状态 (状态: 概括了对过去输入信息处理的状况)
- 系统只需要根据当前所处的状态和当前的输入信息就可以决定系统的后继行为 (下一状态)。每当系统处理了当前的输入后, 系统的内部状态也将发生改变

确定性有穷状态自动机

确定性有穷状态自动机（DFA）的形式定义如下：

$$M = (\Sigma, Q, q_0, \delta, F),$$

其中，

- Σ 是定义的字符集，即处理的输入字符有哪些；
- Q 是有限的状态集合；
- q_0 是初始状态；
- δ 是状态转移函数，形如 $\delta(q, a) = q'$ ，其需要两个输入 q, a ， $q \in Q$ 为当前状态， $a \in \Sigma$ 为当前输入， $q' \in Q$ 为处理输入后的状态；
- $F \subset Q$ 是接受状态集合；

非确定性有穷状态自动机

非确定性有穷状态自动机（NFA）的形式化定义（1）如下：

$$M = (\Sigma, Q, q_0, \delta, F),$$

其中，

- Σ 是定义的字符集，即处理的输入字符有哪些；
- Q 是有限的状态集合；
- q_0 是初始状态；
- δ 是状态转移函数，形如 $\delta(q, a) = q'$ ，其需要两个输入 q, a ， $q \in Q$ 为当前状态， $a \in \Sigma \cup \{\varepsilon\}$ 为当前输入， $q' \in Q$ 为处理输入后的状态；
- $F \subset Q$ 是接受状态集合；

非确定性有穷状态自动机

非确定性有穷状态自动机（NFA）的形式化定义（2）如下：

$$M = (\Sigma, Q, q_0, \delta, F),$$

其中，

- Σ 是定义的字符集，即处理的输入字符有哪些；
- Q 是有限的状态集合；
- q_0 是初始状态；
- δ 是状态转移函数，形如 $\delta(q, a) = \{s_1, s_2, \dots, s_k\}$ ，其需要两个输入 q, a ， $q \in Q$ 为当前状态， $a \in \Sigma$ 为当前输入， $\{s_1, s_2, \dots, s_k\} \subseteq Q$ 为处理输入后的状态；
- $F \subset Q$ 是接受状态集合；

消除不确定性的算法

Algorithm 1: NFA 转 DFA

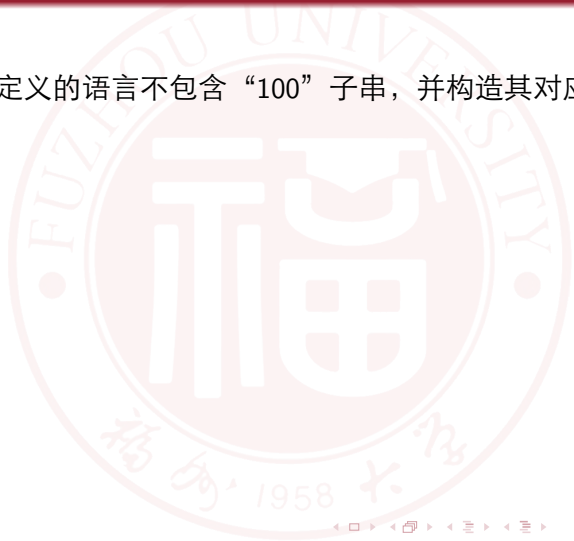
输入: NFA: $(Q, \Sigma, \delta, q_0, F)$

输出: DFA: $(Q', \Sigma, \delta', q'_0, F')$

```
1  $Q' \leftarrow \emptyset;$ 
2  $\delta' \leftarrow \emptyset;$ 
3  $q'_0 \leftarrow \varepsilon\text{-Closure}(\underline{\{q_0\}});$ 
4  $Q'.\text{add}(q'_0);$ 
5 创建未标记状态队列  $workList \leftarrow \{q'_0\};$ 
6 while  $workList$  非空 do
7   从  $workList$  中取出状态  $S;$ 
8   foreach 输入符号  $a \in \Sigma$  do
9      $S' \leftarrow \varepsilon\text{-Closure}(\underline{\text{Move}(S, a)});$ 
10    if  $S' \notin Q'$  then
11       $Q'.\text{add}(S');$ 
12       $workList.\text{add}(S');$ 
13     $\delta'[S, a] \leftarrow S';$ 
14  $F' = \{S \in Q' \mid S \cap F \neq \emptyset\};$ 
```

综合示例

写一个 01 字符的正则表达式，其所定义的语言不包含“100”子串，并构造其对应的 NFA 和 DFA。



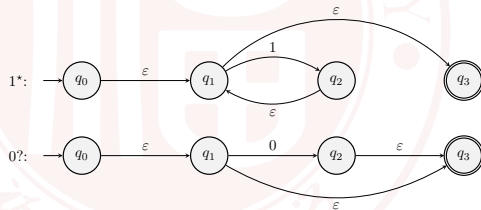
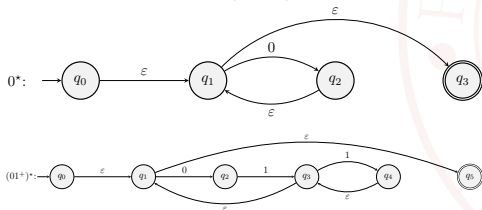
定义的语言不包含“100”子串，并构造其对应

- 1

综合示例

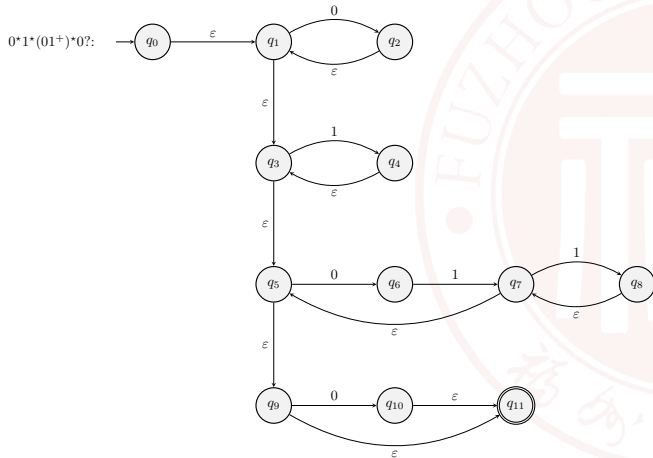
写一个 01 字符的正则表达式，其所定义的语言不包含“100”子串，并构造其对应的 NFA 和 DFA。

- ① 写出正则表达式 $0^*1^*(01^+)^*0?$;
- ② 依次构造 0^* 、 1^* 、 $(01^+)^*$ 、 $0?$ 的 NFA;



综合示例（续）

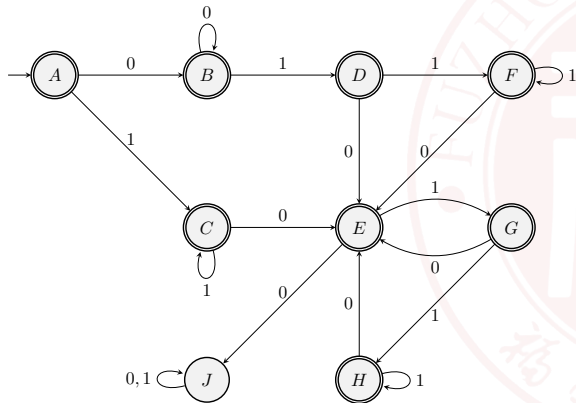
③ 合成 NFA;



④

综合示例（续）

④ NFA to DFA;





① LALR(1) 分析器

② 二义性文法

③ 期中复习

编译程序概论

高级语言的构成

形式化文法

词法分析与正则文法

语法分析与上下文无关文法

语法分析

- 语法分析目的是验证源代码是否符合目标语言的语法规则，并将源代码转换为一种结构化的表示形式（如抽象语法树）。
- 语法分析器（Parser）通过分析词法分析器生成的词法单元序列，确保程序的语法正确性，并为后续的语义分析和代码生成提供基础。
- 语法分析的基础是**上下文无关文法**。

上下文无关文法 $G = (N, \Sigma, P, S)$, 其中文法

- 产生式规则集 P , 每个规则都为如下形式

$$N \rightarrow (\Sigma \cup N)^*,$$

即上下文无关文法的产生式左侧仅允许存在一个非终结符。

下推自动机 (Pushdown Automaton, PDA)

- 下推自动机，是一个抽象的机器用于识别和接受字符输入。
- 它和非确定型有限自动机类似，但多出一个称为栈（stack）的组件，且栈的规模是无限的。
- PDA 默认都是非确定性的，确定性的和非确定性的不能等价。

一个 PDA 可以被定义为六元组

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F),$$

其中,

- Q 是所有状态的集合, 有限个;
- Σ 是输入字符的集合, 有限。特别地, $\Sigma_\epsilon = \epsilon \cup \Sigma$
- Γ 是栈字符的集合, 有限。特别地, $\Gamma_\epsilon = \{\epsilon\} \cup \Gamma$
- δ 是转移函数

$$\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow Q \times \Gamma_\epsilon$$

根据输入和当前栈顶元素进行转移的同时替换栈顶元素特别地, 转移函数支持 ϵ 作为栈顶元素, 也支持将栈顶元素替换为 ϵ 。这两个操作一个是单纯压栈, 另一个是单纯弹栈 q_0 , F 的意思和 NFA 中的一致

语法分析器的类型

自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从根开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从输入串开始，逐步进行归约，直到文法的开始符号。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。
- 从叶节点开始，构造语法树。
- 具体实现：算符优先分析法、LR 分析法。

Top-Down 分析器挑战一：左递归

在自顶向下分析器寻找匹配的推导时，通常采用的是最左推导的形式，即每次选择**最左边**的非终结符进行替换。

按顺序尝试非终结符的不同产生式，有可能出现最左侧符号不断出现的**无限循环**的情况。

文法

目标句子：8+8+8

$expr \rightarrow expr + term \mid expr - term \mid term$

$term \rightarrow term * factor \mid term / factor \mid factor$

$factor \rightarrow digit \mid (expr)$

$digit \rightarrow 0 \mid 1 \mid \dots \mid 9$

$expr \Rightarrow expr + term$

$\Rightarrow expr + term + term$

$\Rightarrow expr + term + term$

$\Rightarrow expr + term + term + term$

$\Rightarrow expr + term + term + term + term$

Top-Down 分析器挑战一：左递归

左递归的两种形式:

直接左递归

$$expr \rightarrow expr + term | expr - term | term$$
$$term \rightarrow term * factor | term / factor | factor$$
$$factor \rightarrow digit|(expr)$$
$$digit \rightarrow 0|1|\dots|9$$

间接左递归

$$S \rightarrow AB$$
$$A \rightarrow TC$$
$$T \rightarrow S_\gamma$$

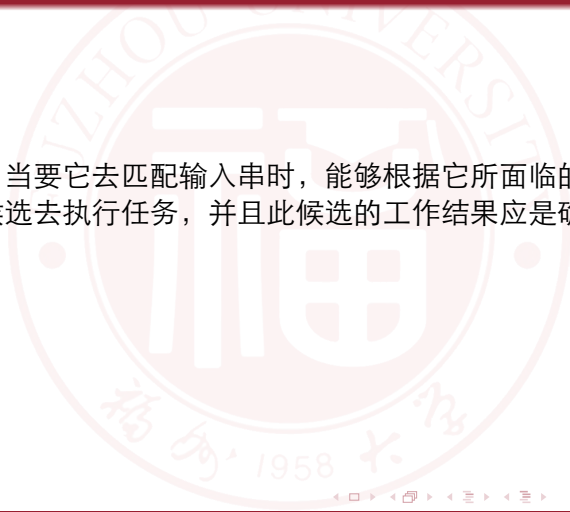
Top-Down 分析器挑战二：回溯

- 分析过程中，当一个非终结符用某一个候选式匹配成功时，这种匹配可能是暂时的。出错时，需要进行“回溯”。
- 当同一个非终结符的多个候选式存在共同前缀时，将导致回溯。

Top-Down 分析器挑战二：回溯

消除回溯

- 目标：对文法的任何非终结符，当要它去匹配输入串时，能够根据它所面临的输入符号准确地指派它的一个候选去执行任务，并且此候选的工作结果应是确信无疑的。
- 方法：提取左公因子



数量优先

改写文法 $S \rightarrow abcd|abce|abf$ 以便无回溯的自顶向下分析。

S 所有子式的最长左公因子为 ab , 引入 A 改写生成式得 $S \rightarrow abA, A \rightarrow cd|ce|f$ 。
再提取 A 的左公因子并引入 B 改写生成式得 $A \rightarrow cB|f, B \rightarrow d|e$ 。
最终文法为

$$S \rightarrow abA$$

$$A \rightarrow cB|f$$

$$B \rightarrow d|e$$

文法处理的顺序

- ① 消除左递归;
- ② 提取左公因子以消除回溯;
- ③ 处理文法的二义性;

递归下降分析法

典型过程

Algorithm 2: 处理非终结符号的典型过程

```
1 void A () {  
2   选择一个 A 的产生式,  $A \rightarrow X_1, X_2, \dots, X_k$ ;  
3   for  $i \leftarrow 1$  to  $k$  do  
4     if  $X_i$  是一个非终结符号 then  
5       调用过程  $X_i()$ ;  
6     else  
7       if  $X_i$  等于当前的输入符号  $a$  then  
8         继续读取下一个输入符号;  
9       else  
10        处理错误;  
11      end  
12 end
```

FIRST

定义

FIRST(α) 是可以由 α 推导得到的串首符号的集合, 其中 α 是任意的文法符号串, 具体地

- $\forall \alpha \in (\mathbb{V}_T \cup \mathbb{V}_N)^+, \text{FIRST}(\alpha) = \{a | \alpha \Rightarrow^* a\beta, a \in \mathbb{V}_T, \beta \in (\mathbb{V}_T \cup \mathbb{V}_N)^*\}$
- 若 $\alpha \Rightarrow^* \varepsilon$ 合法, 则 $\varepsilon \in \text{FIRST}(\alpha)$

推论

如果非终结符 A 的所有候选式的首终结符集两两不相交，当要求 A 匹配输入串时， A 就能根据当前输入符号 c ，选择唯一的正确的候选式，即串首终结符集中包含 c 的那个候选式。

FOLLOW

- 对于非终结符号 A , $\text{FOLLOW}(A)$ 是所有在某些句型中紧跟在 A 右边的终结符号的集合。
- A 与 a 之间可能存在其他的非终结符, 但都会随着推导变成 ε 。
- 特别地, 如果 A 是某些句型最右的符号, 则 $\$ \in \text{FOLLOW}(A)$, 其中 $\$$ 表示输入的终止 (EOF)。

例

如果存在 $S \Rightarrow^* \alpha A a \beta$ 的推导, 则 $a \in \text{FOLLOW}(A)$ 。

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ϵ 边作为最后的默认选择以解决 ϵ 的二义性（例如，if 语句的 else 匹配问题）。

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ε 边作为最后的默认选择以解决 ε 的二义性（例如，if 语句的 else 匹配问题）。

LL(1) 文法

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ε 边作为最后的默认选择以解决 ε 的二义性（例如，if 语句的 else 匹配问题）。

定义

- LL(1) 文法是一类特别的上下文无关文法。
 - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
 - 第二个 L 表示 “Leftmost”，即最左推导；
 - 1 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 左递归文法和二义文法都不是 LL(1) 文法。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- LL(1) 文法可以将 ϵ 边作为最后的默认选择以解决 ϵ 的二义性（例如，if 语句的 else 匹配问题）。

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。

表驱动的预测分析法

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- **小提问：**对于每一个 NFA 都可以构造一个 DFA 与其等价，那所有的 PDA 都可以构造一个确定性的下推自动机与其等价吗？

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。

定义

- **预测分析法**是一种表驱动的方法，它由下推栈，预测分析表和控制程序组成，是一种确定性的下推自动机。
- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。
- 要求：文法是 LL(1) 的。只有对于 LL(1) 文法，可以对输入串进行有效的无回溯的自上而下分析。

预测分析表

定义

- **预测分析表**是用于自顶向下语法分析（如 LL(1) 分析器）的核心工具，它通过非终结符和输入符号的匹配，指导分析器选择正确的产生式，避免回溯。
- 预测分析表是一个二维表格（矩阵）：
 - 行代表每一个非终结符；
 - 列代表每一个终结符（包括结束符号 \$）；
 - 每个单元格里填写对应的产生式，或者错误标记；
 - 单元格 (i, j) 的内容表示若在匹配第 i 行的非终结符时遇到第 j 列的终结符时，该确定性地选用的生成式。

写出文法的预测分析表:

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

例

写出文法的预测分析表:

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

推论

如果冲突的是生成 ϵ 的产生式, 那么可以将 ϵ 产生式丢弃。

预测分析表

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

例

基于上面的预测分析表，尝试解析 $(id * id) + id - id * id$ 。